

Parent Learning Tool — Design

Guri Dagan · draft for review · not yet built

1. What problem this solves

Right now, every piece of parenting advice mom gives happens live, 1-on-1, over WhatsApp — typed out fresh, every time, for every parent, even when the same questions repeat (discipline without hitting, screen time, sibling fights, tantrums). That doesn't scale past whatever hours she can personally give, and a parent who got good advice on Monday has nothing to go back to on Friday.

The learning tool turns recurring advice into a small library of reusable lessons a parent can receive and revisit on WhatsApp — without mom re-typing the same guidance each time, and without asking parents to log into a new app.

Correction from earlier framing: the first draft of this design tied lessons to the 5 formal coaching programs. Most clients are just parents reaching out with a problem — they are not necessarily enrolled in one of the 5 programs. So lessons are organized by **topic** (the actual parenting problem), not by program. A program tag is optional metadata for the minority of clients who are enrolled, not the primary structure.

2. Where it lives (fits into what already exists)

- Delivery channel: **WhatsApp**, same bot as today (`app/api/whatsapp/route.ts`) — no new client-facing app or login.
- Drafting engine: **Gemini**, same client already used for advice replies (`lib/gemini.ts`) — same warm, plain Somali tone, same short-reply style.
- Send timing: reuses the existing **delayed-send queue** (`whatsapp_pending_replies`) so lessons don't arrive instantly/robotically.
- Who a lesson goes to: existing **leads** table (phone, stage, source) — a lesson send is just another kind of outbound message tied to a lead/client record.

Nothing about the WhatsApp bot's existing intake or advice-reply flow changes. This is a new, separate layer: a library of reusable content, and a way to schedule it out.

3. Content model — organized by topic, not program

A **lesson** is a short, WhatsApp-native message in Somali (4–8 lines, same length discipline as current advice replies) covering one practical idea a parent can act on. Lessons are grouped under **topics** — the actual recurring problems parents bring, e.g.:

Topic	Example lessons in it
Edbin la'aan garaacid (discipline without hitting)	3–4 short lessons, each one alternative approach
Xanaaq & qarax (tantrums/outbursts)	naming the feeling, cool-down steps, after-the-storm talk

Isticmaalka mobile-ka (screen time)	limits without a fight, replacement activities
Khilaaf walaalo (sibling conflict)	fair-not-equal, referee vs. coach
Xiriirka waalid-ilmo (connection/communication)	daily check-in habits, listening without fixing

A lesson can *optionally* carry a program tag (for the enrolled minority), and can optionally target a child-age range, since the WhatsApp intake already captures track (parent/child) and child age.

4. How a lesson gets written

1. Mom picks (or the system suggests) a **topic** — e.g. from questions that keep recurring in WhatsApp conversations.
 - 2. Gemini drafts a short lesson in that topic, same system-prompt style already tuned for Somali advice replies.
 - 3. Draft lands in a review queue — **mom reads, edits, approves**. Nothing reaches a parent unreviewed.
 - 4. Once approved, the lesson is "published" and becomes eligible to send.

This keeps mom as the quality gate — AI drafts the first pass so she isn't typing from a blank page, but nothing goes out to a real parent without her okay.

5. How a lesson reaches a parent

Two ways in, so it works for both the drop-in question-asker and the parent trying to genuinely learn over time:

a) Matched to what they just asked about

After the bot gives a live advice reply on a topic (say, tantrums), it can queue a related published lesson to arrive a day or two later — reinforcement, not instant info-dump.

b) A light drip, if they opt in

A parent who wants to "keep learning" gets one lesson every few days from topics relevant to their situation (child's age, whatever they've asked about before) — until they pause it or run out of relevant lessons.

Open decision: should the drip start automatically after someone qualifies as a lead, or only if they explicitly ask ("send me more like this")? Recommendation: opt-in only at first — automatic drip to everyone risks feeling spammy before we know engagement is any good.

6. Tracking whether it's working

- Per send: which lesson, which parent, when sent.
- Simple engagement signal: did they reply to it at all (yes/no) — full sentiment analysis is out of scope for v1.
- Visible on the client's existing detail page (`/clients/[id]` or `/leads/[id]`) as a small "Lessons sent" list, so mom has context next time she talks to that parent.

7. Mom's side of the tool (admin view)

- A library view grouped by topic: draft / published lessons, edit inline, publish/unpublish.
- A "generate lesson" action: type a topic → AI draft appears → edit → publish.
- Per-client: a "Lessons sent" panel + a manual "send this lesson now" override for when a conversation calls for something specific right away.

8. Data shape (for reference — not final schema)

Table	Holds
lessons	topic, body (Somali), status (draft/published), optional program tag, optional age range
lesson_sends	lesson id, lead/client id, sent_at, replied (bool)

Delivery itself reuses the existing `whatsapp_pending_replies` queue rather than inventing a second send mechanism.

9. Suggested rollout order

Phase 1 — Library + manual send. Mom builds/approves lessons, sends them by hand from a client's page. No automation yet. Proves the content is good before anything runs on its own.

Phase 2 — Topic-matched follow-up. Bot auto-queues a relevant lesson after it gives live advice on that topic.

Phase 3 — Opt-in drip for parents who want ongoing lessons, plus reply-based engagement tracking.

10. Open questions before building

1. Opt-in drip vs. automatic — see decision in section 5.
2. How many lessons before a topic is "enough" to launch — 3–4 per topic to start, or fewer?
3. Should a parent be able to say "stop" / "send me something else" mid-conversation, and does the bot need to recognize that?

Design draft — no code written yet. Next step once this is approved: confirm schema + build Phase 1.